Waiting for Everyone to Join

Féidearthachtaí as Cuimse
Infinite Possibilities

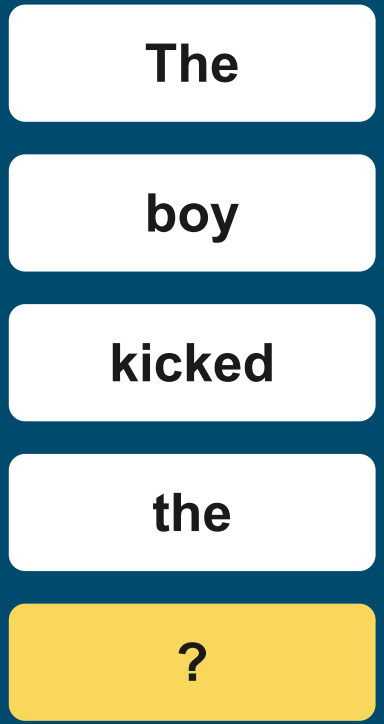
Natural Language Technologies
Muhammad Abeer

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

02/11/26

Language Models

How machines learn to predict the next word?

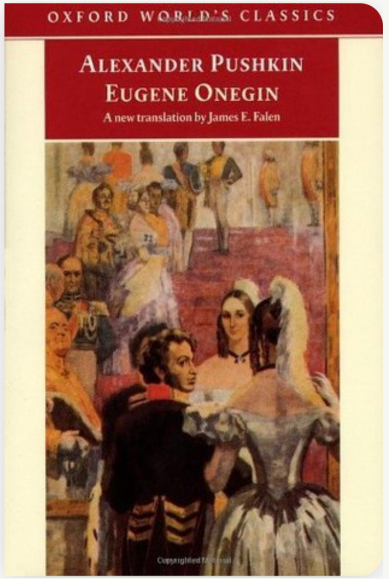



Once Upon
a Time

Once Upon a Time…

…in St Petersburg in 1913, a mathematician picked up the most famous poem in the Russian language: Pushkin's Eugene Onegin.

He was not reading it for pleasure.



Once Upon a Time…

He crossed out every space and punctuation mark…

…and counted the first **20,000 letters**, by hand.

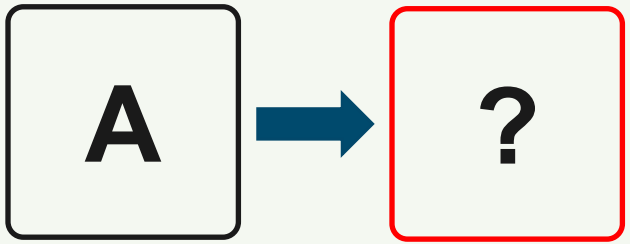


Мой дядя самых честных правил

Once Upon a Time…

He asked one simple question:

If this letter is a vowel, what is the next letter likely to be?



Once Upon a Time…

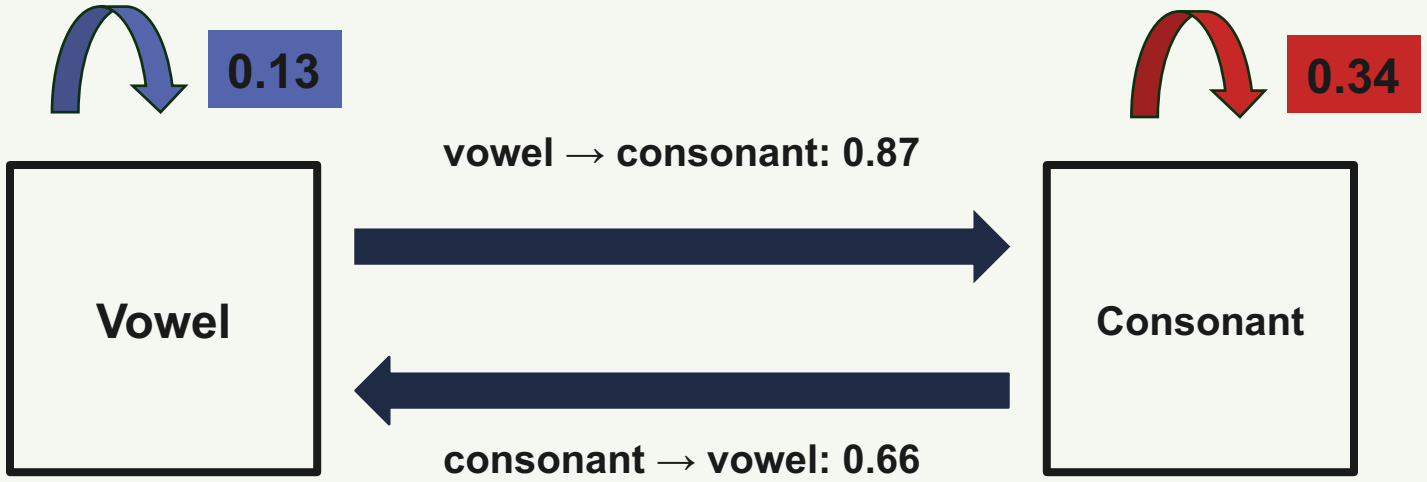
His idea is beautifully simple: *what comes next depends on what came just before.*

His name was **Andrey Markov**.

And his idea is called the **Markov chain**.

Every language model you use today, including ChatGPT, is its descendant.

What Markov found?

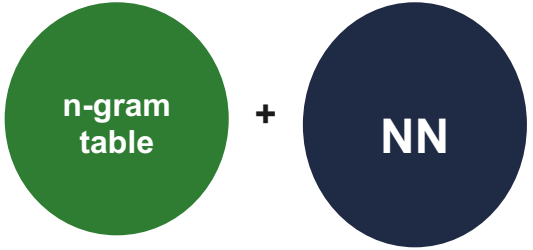


After a vowel, a vowel only 13% of the time

Source: Markov (1913), as reported by Hayes, American Scientist (2013) and IEEE Spectrum (2019)

N-grams are back

Researchers at DeepSeek-AI and Peking University added a giant n-gram lookup table to a modern LLM, so it can look up common word patterns instead of recomputing them.



Engram

Today's idea, inside tomorrow's models.

Cheng et al. (2026) Conditional Memory via Scalable Lookup, arXiv 2601.07372

What are we doing today? (more formal)

- Define statistical language models and their two core tasks: next-word prediction and sequence scoring.
- Compute unigram and bigram maximum-likelihood probabilities and apply the chain rule to sentences.
- Explain data sparsity, smoothing and interpolation.
- Calculate and interpret perplexity on held-out text.
- Contrast statistical and neural language models.

Where have we heard about this before?

We already know more about this than we think…

Everyday life



Phone keyboard suggestions



Search autocomplete



Smart replies in email

Earlier in this module



Week 4: tokens and bag of words



Week 5: classical NLP pipeline



Week 6: the BBC news corpus

Activity: Be the Language Model

Activity: Be the Language Model

He sat with the ring in his pocket, waiting for the perfect moment. Finally, he slid his hand from his pocket and asked Marie if she would…



What is the next word?

Which clues did you use?

Big Idea: you just predicted the next word



A language model does the same, using probabilities learned from text.

Key Terms

Language model: a model that gives the probability of a sequence of words. *e.g. how likely is “she went to the shops”?*

Corpus: the collection of text a model learns from. *e.g. 2,225 BBC news articles*

Token: one unit of text, usually a word or part of a word. *e.g. “the”, “cat”, “un” + “believ” + “able”*

N-gram: a chunk of N tokens in a row. *e.g. “the cat” is a 2-gram (bigram)*

A language model does two jobs

1. What comes next?

Mary …
Mary had …
Mary had a little …

2. Which is more likely?

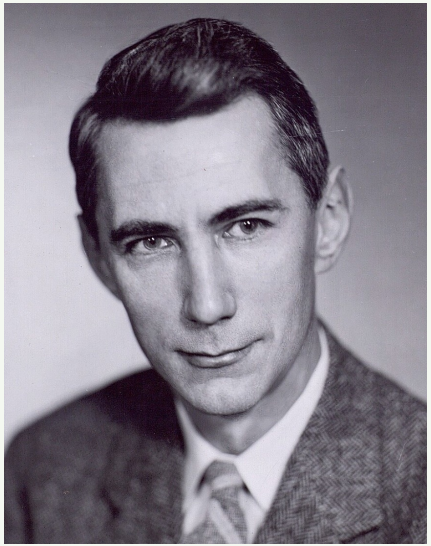
She went to the shops.

She went two the shops.

35 years later…

…at Bell Labs in New Jersey, an engineer wondered how predictable English really is.

His name was **Claude Shannon**, and in 1948 he made statistics write.



Source: Wikipedia

More context, more English

Random letters

```
XFOML RXKHRJFFJUJ ZLPWCFWKCYJ FFJEYVKCQSGHYD
```

Letter frequencies

```
OCRO HLI RGWR NMIELWIS EU LL NBNESEBYA TH EEI
```

Previous two letters

```
IN NO IST LAT WHEY CRATICT FROURE BIRS GROCID
```

Previous word

```
THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER
```



Activity: The Shannon Game

Activity: The Shannon Game

Guess the next word and at for the last word guess the next letters.

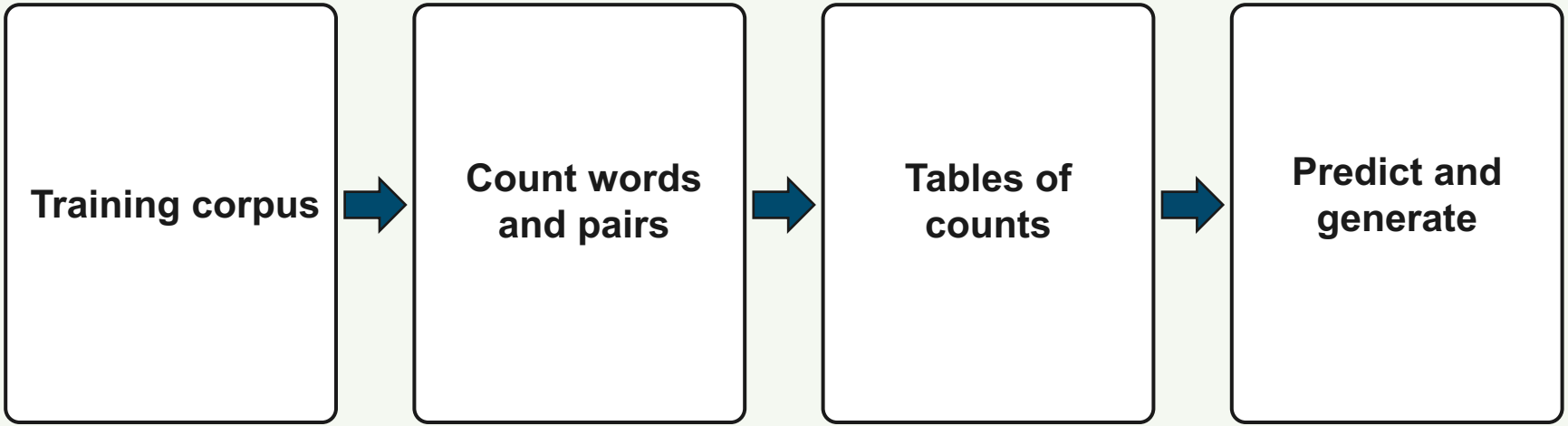


English is highly predictable.

Counting Words

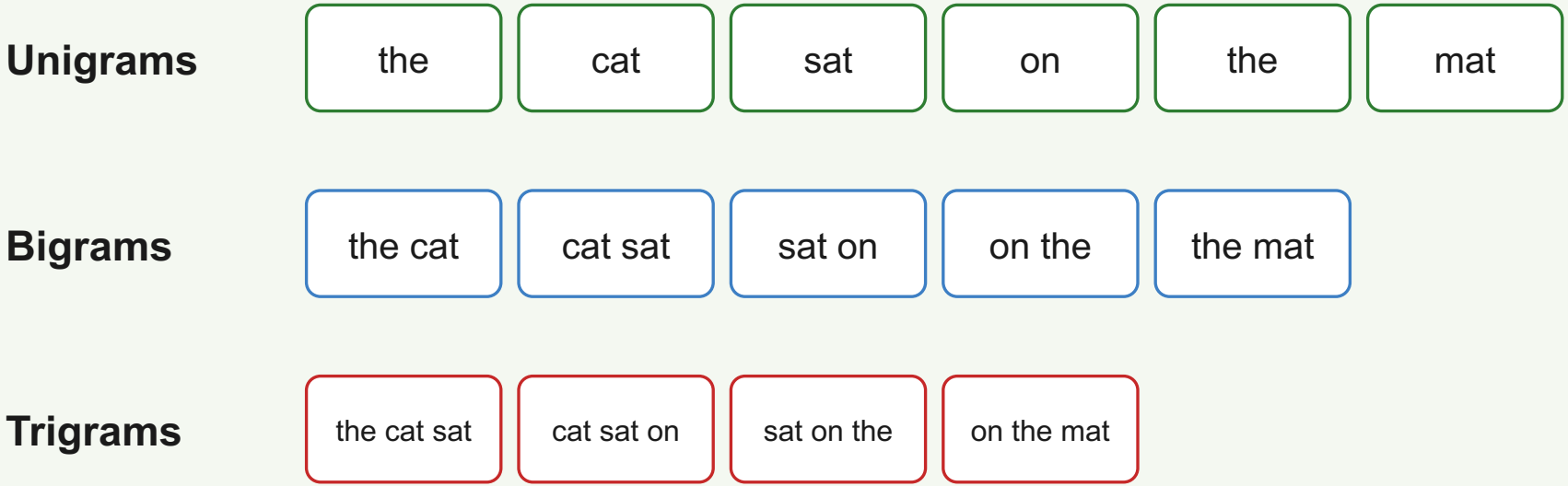
Statistical language models

A statistical model is a table of counts



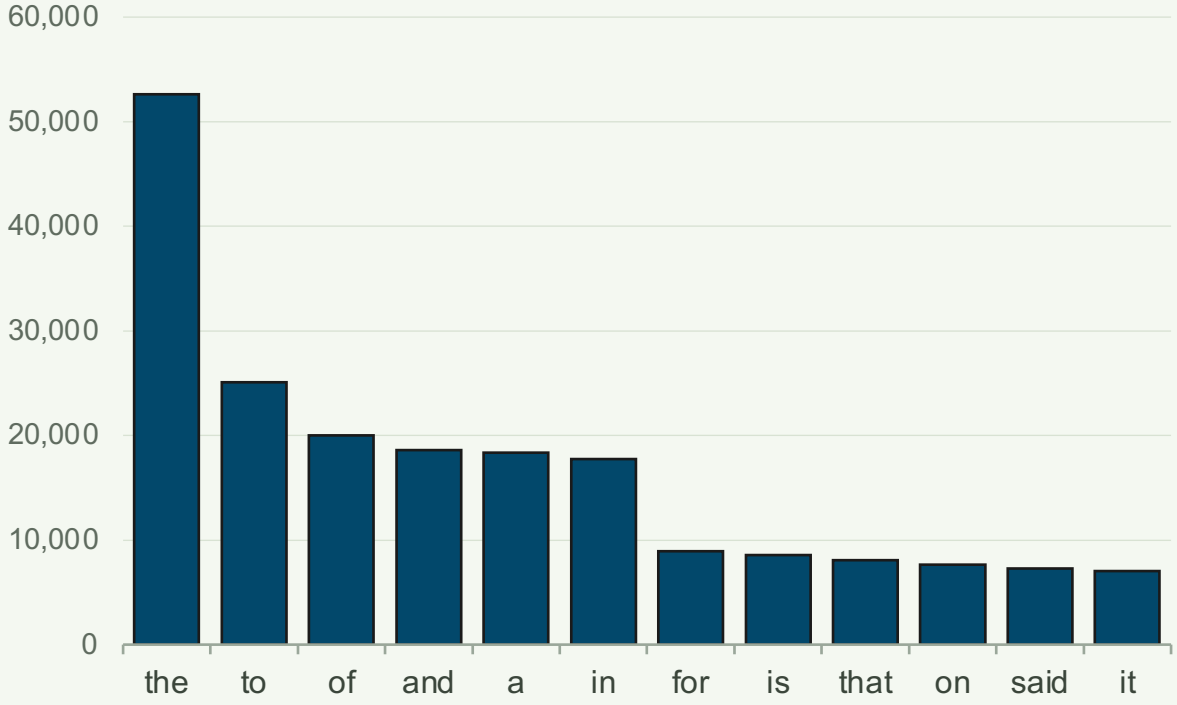
No neural network. Just counting.

N-grams: chop the text into chunks



N = how many words are in each chunk

A few words do most of the work



6.2%

of every word in the BBC corpus is “the”

35%

of distinct words appear only once

https://www.youtube.com/watch?v=jRa3okU4X1o

Key Terms

Unigram model: every word is scored on its own. *e.g. P(“ball”)*

Bigram model: every word depends on the word before it. *e.g. P(“ball” | “the”)*

Conditional probability: how likely something is when we already know something else. *e.g. P(next word | previous word)*

Maximum likelihood: turn counts into probabilities by dividing. *e.g. count(of the) / count(of)*

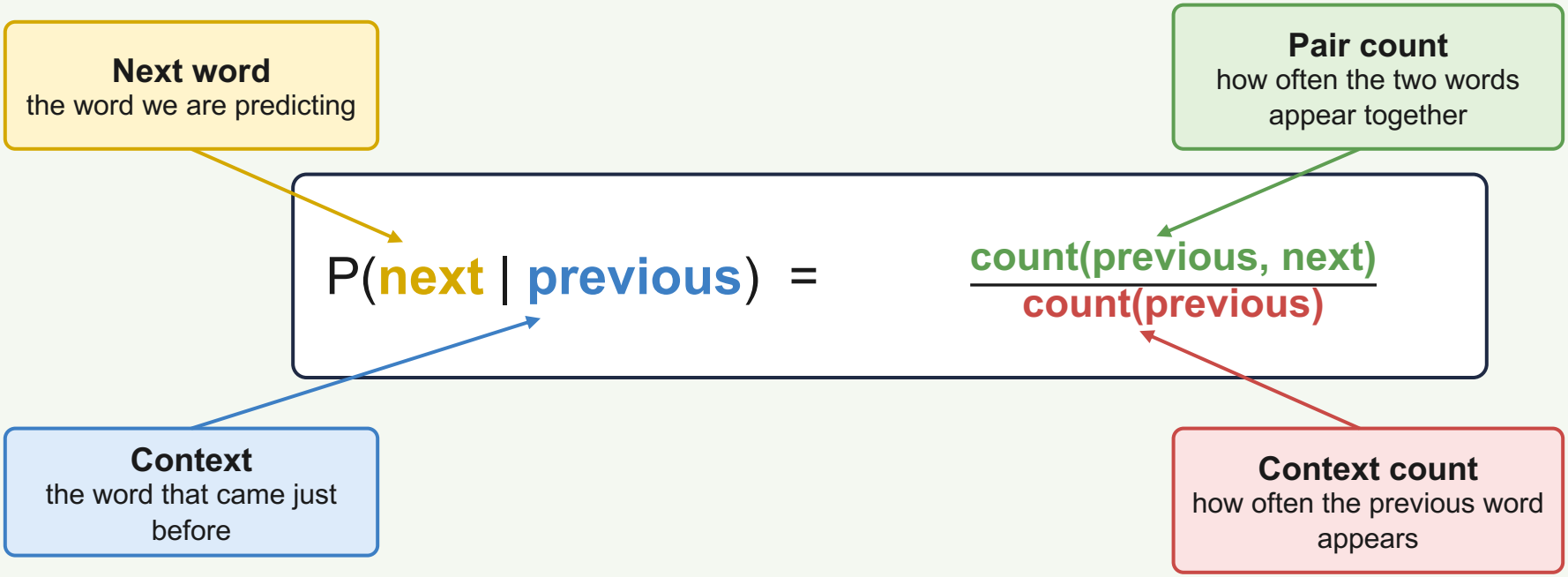
Unigram model: every word on its own

$$P(w_1\ w_2\ \ldots\ w_n) = P(w_1) \times P(w_2) \times \ldots \times P(w_n)$$

the boy kicked the ball **=** *ball the kicked man the*

Same score! Word order is ignored.

The Bigram Formula



Real numbers: what follows “of”?

count(of the)
4,996

÷

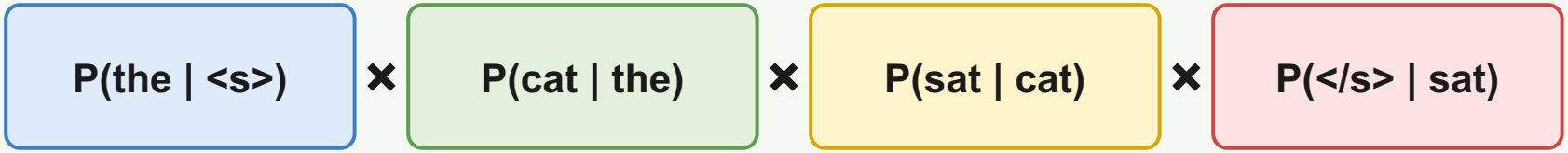
count(of)
20,008

= **25%**

One in four times, the word after “of” is “the”.

Example: Chain Rule (Part 1)

A sentence is a chain of next-word events:



<s> marks the start of a sentence, </s> the end.

P(the cat sat) = the product of the chain

Example: Chain Rule (Part 2)

```
the cat sat
the cat ate
the dog sat
```

Training corpus

P(the | <s>) = 1

P(cat | the) = 2/3

P(sat | cat) = 1/2

P(</s> | sat) = 1

1 × 2/3 × 1/2 × 1 = 1/3

So there is a 33% chance this model produces “the cat sat”.

Activity: Bigram Detective

Activity: Bigram Detective

use this corpus:

```
the cat sat
the cat ate
the dog sat
```

1. How many times does “the” appear?
2. What is P(cat | the)?
3. What is P(sat | cat)?
4. **Challenge: what is P(the dog ate)?**

Activity: Bigram Detective

1. “the” appears 3 times.

2. P(cat | the) = 2/3 (the cat, the cat, the dog)

3. P(sat | cat) = 1/2 (cat sat, cat ate)

4. P(the dog ate) = 1 × 1/3 × **P(ate | dog)**

P(ate | dog) = 0, so the whole sentence = 0 !!!

Activity: How Many Word Pairs?

Activity: How Many Word Pairs?

The BBC corpus has **30,494** different words.

How many different word pairs (bigrams) are possible?

A. About 30 thousand

B. About 1 million

C. About 930 million

D. About 30 billion

Activity: How Many Word Pairs?

Vocabulary	Possible bigrams
10 words	100
1,000 words	1,000,000
30,494 words (BBC)	**929,884,036**

Bigrams actually seen

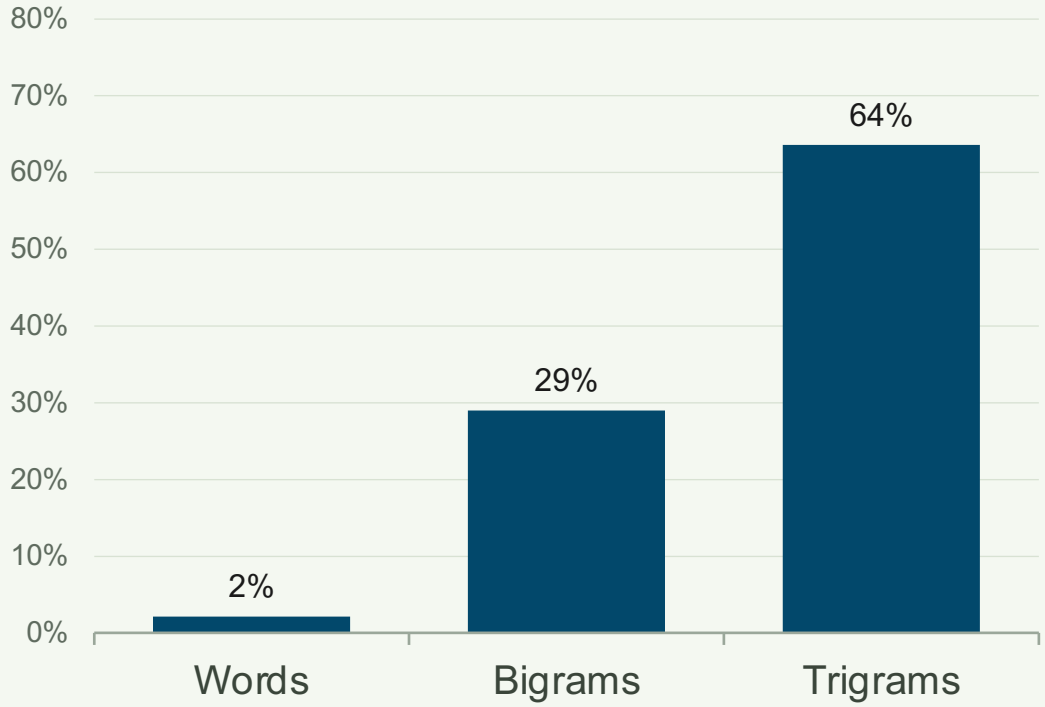
322,197

0.03%

of all possible pairs

Most word pairs are never seen. Not because they are wrong, but because the data is finite.

The more context, the more we have never seen



Test articles the model never saw

64%

of test trigrams were never seen in training

BBC corpus, 90/10 train-test split

Fixing the zeros

1. Smoothing

Pretend every pair was seen once more.

count(dog ate) **0** → **1**

Nothing is impossible any more.

2. Interpolation

Mix long and short contexts.

trigram × 0.5

bigram × 0.3

unigram × 0.2

Back off when the long context is unseen.

Example: The Cupertino Effect

Spell checkers with no sense of context “corrected” a common word into a place name. These phrases appeared in official documents:

South Asian Association for Regional Cooperation



South Asian Association for Regional Cupertino

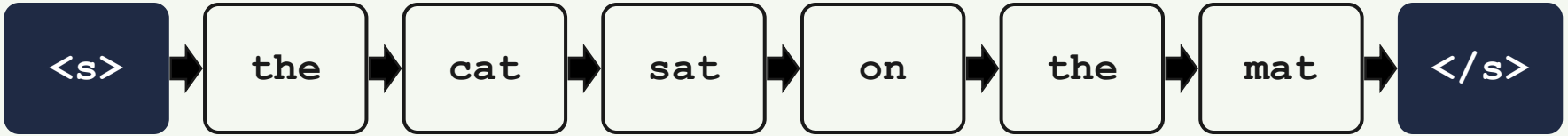
Source: Cupertino effect, reported in New Scientist (2007) and Language Log (2006)



Generating Text

Prediction on repeat

Generation is prediction on repeat

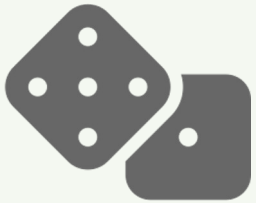


Pick a likely next word. Repeat.

Don’t always pick the top word

The boy kicked the ___





Sampling = rolling a loaded die

Likely words often, surprising words sometimes

How Good Is Our Model?

Key Terms

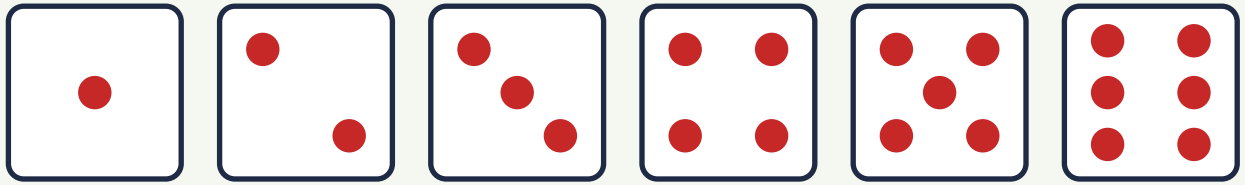
Perplexity: how surprised a model is by new text. Lower is better. *e.g. a fair die has perplexity 6*

Held-out test set: text kept aside, never used for training. *e.g. 10% of the BBC articles*

Smoothing: giving unseen n-grams a small probability. *e.g. add-one smoothing*

Interpolation: mixing n-gram models of different lengths. *e.g. 0.5 trigram + 0.3 bigram + 0.2 unigram*

A fair die has perplexity 6



= 6

Perplexity = the number of equally likely choices the model is torn between.

Lower perplexity = less surprised = better model.

The Perplexity Formula

Probability of the test text
how likely the model finds the unseen text

$$PP(W) = P(w_1\ w_2\ \ldots\ w_n)\ \hat{}\ (-1/N)$$

Invert and take the Nth root
high probability gives low perplexity; fair to long texts

Lower is better
train on one text, test on another

Activity: The Surprise Meter

Activity: The Surprise Meter

A model always chooses evenly between 4 possible next words.

What is its perplexity?

A. 1	B. 2	**C. 4**	D. 16

Four equally likely choices = perplexity 4

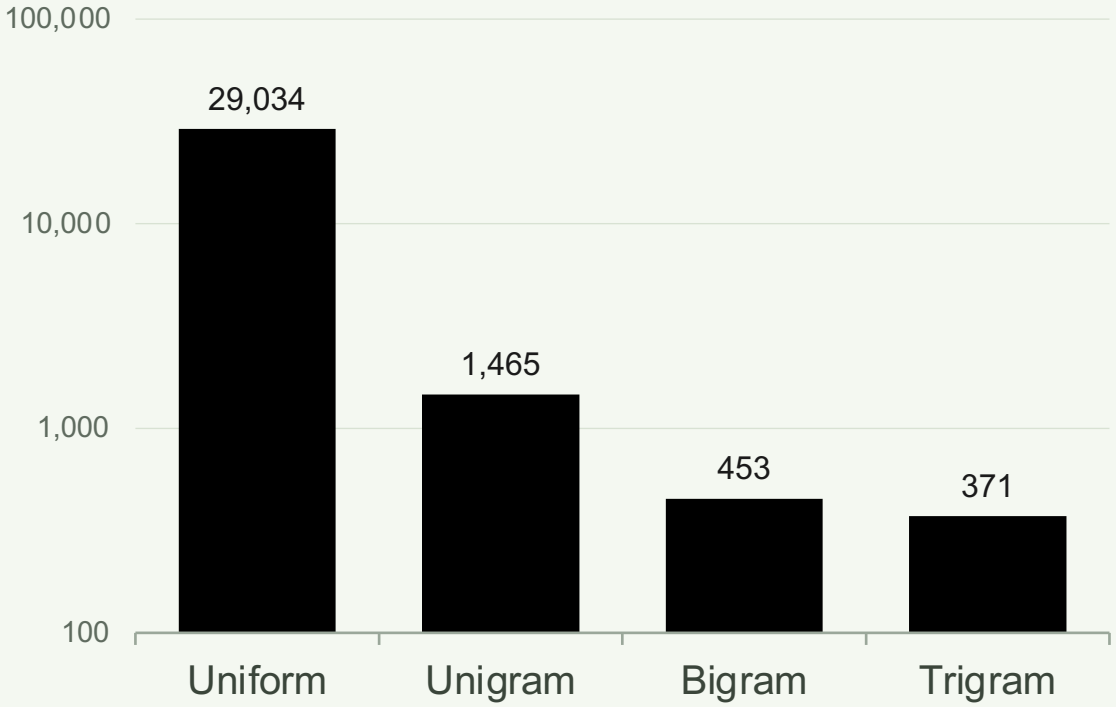
Activity: Calculate a Perplexity

```
Corpus:  the cat sat on the mat
```

Using a unigram model, what is the perplexity of “the cat”?

P(the) × P(cat)	= 2/6 × 1/6 = 1/18
N = 2 words	PP = (1/18)^(−1/2)
Answer	**= √18 ≈ 4.24**

Real results: more context, less surprise



From 29,034 to 371

just by remembering the previous two words

Log scale. BBC corpus, 90/10 split, computed for this lecture

Beyond Counting

Neural language models

Counts don’t know cats and dogs are alike

the cat sat on the mat

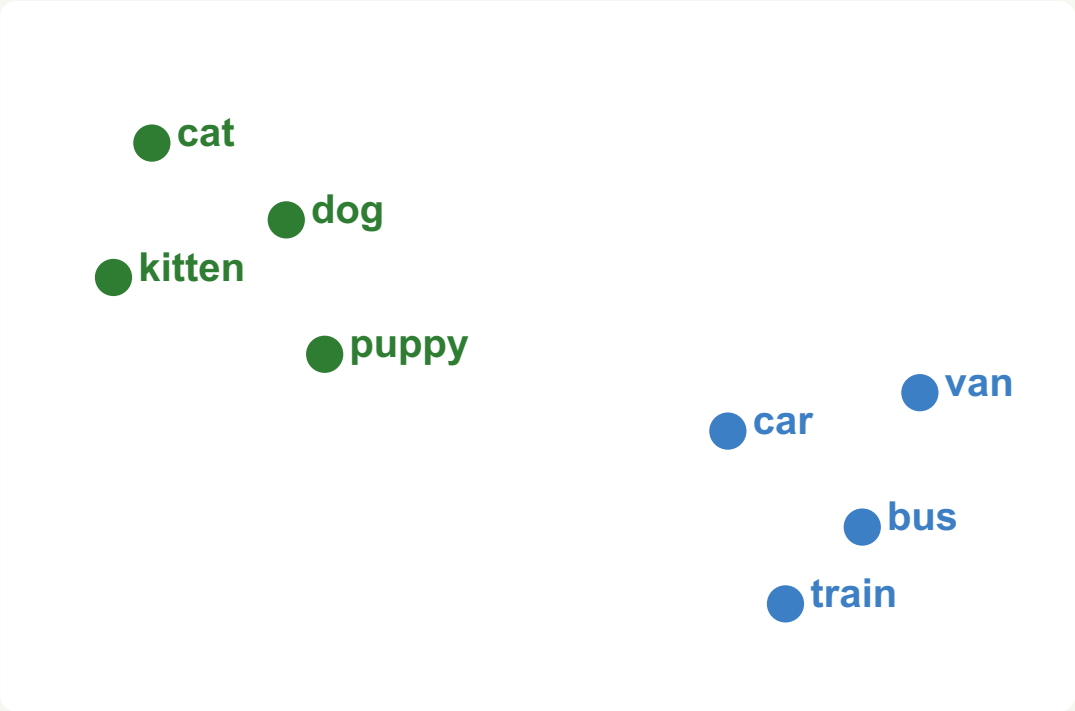
seen in training

the dog sat on the rug

never seen, so the counts say: impossible

A neural model learns that cat ≈ dog.

Words as points in space



Illustrative 2-D picture of word embeddings

Similar words, similar vectors.

So “the dog sat” borrows evidence from “the cat sat”.

Key Terms

One-hot vector: a long list of zeros with a single 1 that says which word it is. *e.g. “kicked” = [0, 0, 1, 0, 0, 0]*

Word vector (embedding): a short list of learned numbers that places a word in space. *e.g. cat and dog get nearby vectors*

Hidden layer: a layer of the network that mixes its inputs and finds useful patterns. *e.g. combines “girl kicked the” into one summary*

Softmax: turns raw scores into probabilities that add up to 1. *e.g. ball 0.10, boy 0.08, bucket 0.05 …*

Step 1: a word becomes a one-hot vector



A computer cannot read words, only numbers.

So, every word gets a list of zeros with a single 1 in its own place.

Real vocabulary: 50,000 words
So, each vector is 50,000 numbers long

Step 2: one-hot becomes a word vector

one-hot for “kicked”
50,000 numbers, almost all 0



word vector for “kicked”
[0.7, −0.2, 0.5, …]
only about 100 numbers

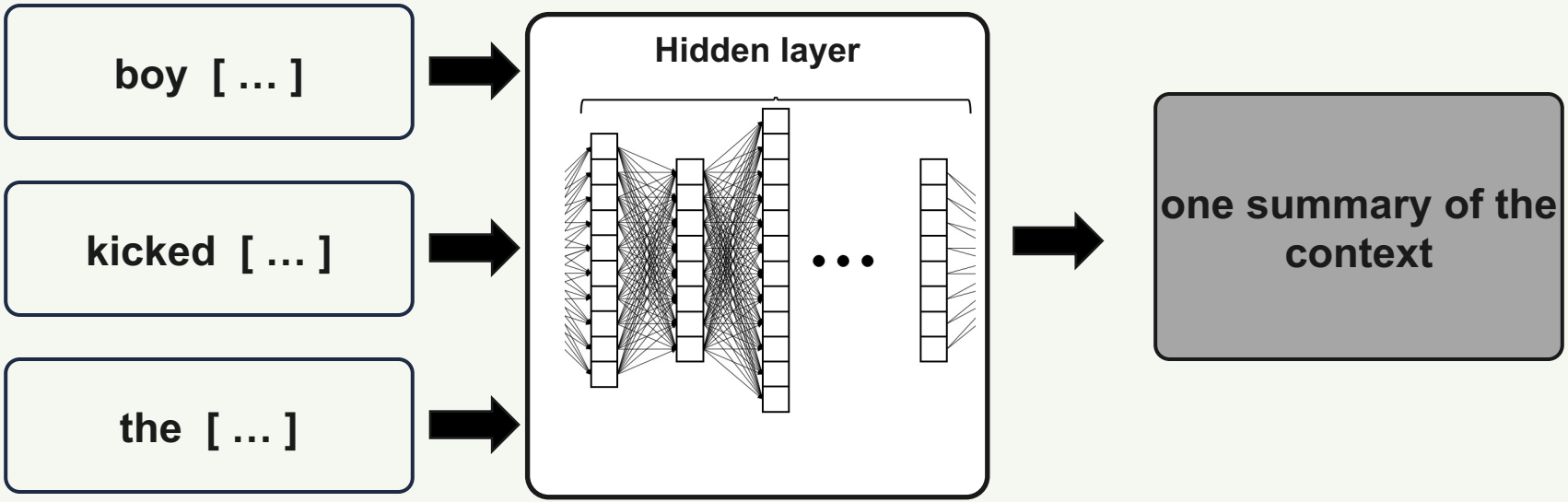
The numbers come from a lookup table that the network learns during training.

cat [0.8, 0.1, 0.6]

dog [0.7, 0.2, 0.5]

Words used in similar ways end up with similar numbers.

Step 3: the hidden layer mixes the words



Step 4: softmax gives every word a probability

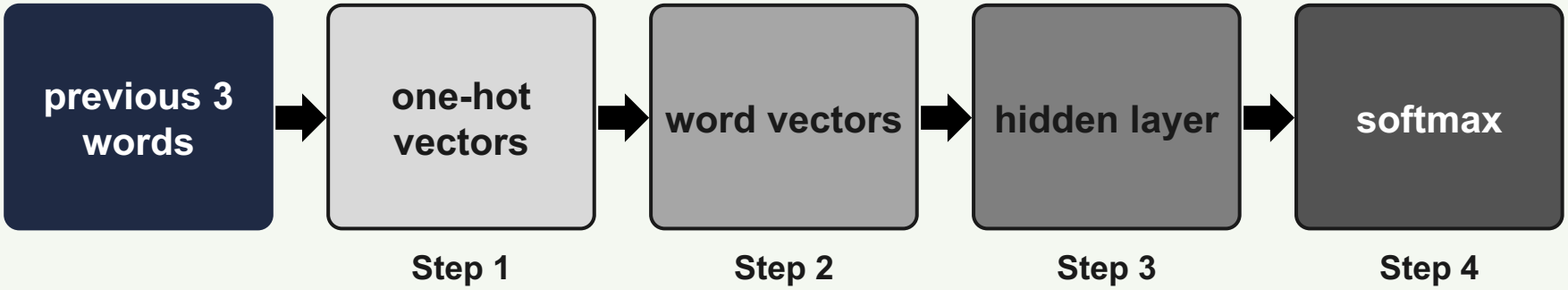


… and thousands more words share the rest

Softmax
turns the network's raw scores into probabilities that add up to 1

Pick one
take the top word, or sample from the distribution

The whole early neural language model



Probability of every possible next word

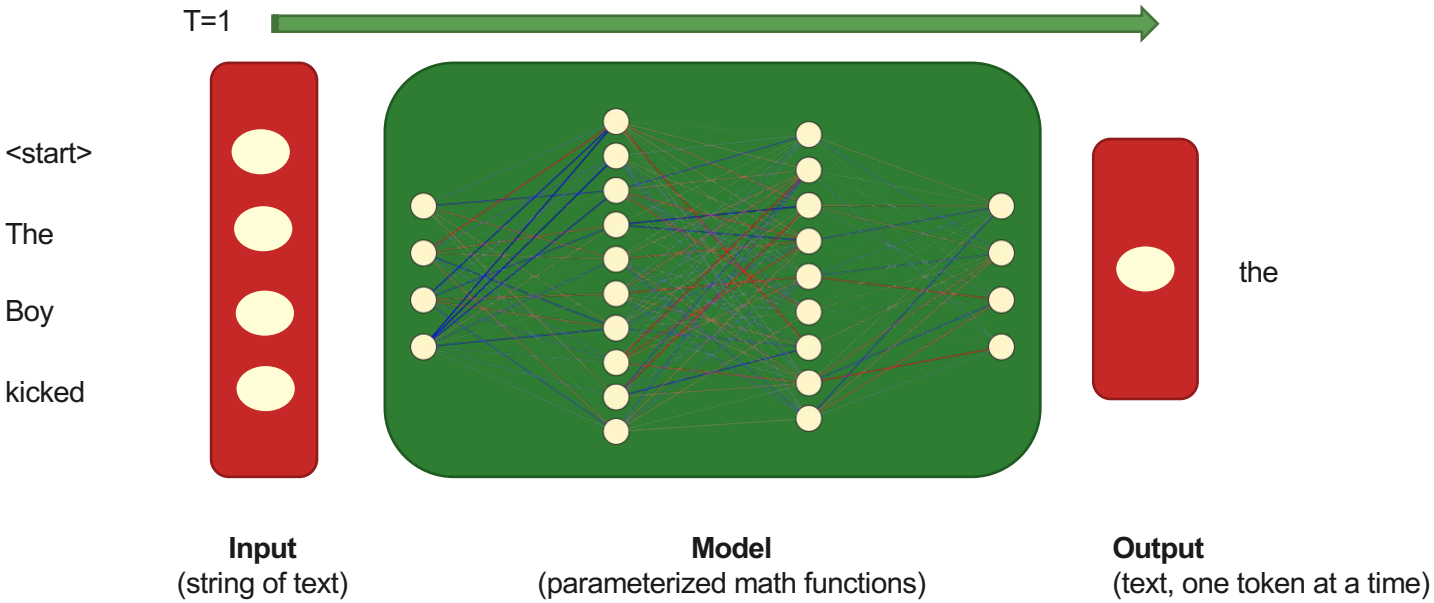
Bengio, Ducharme, Vincent and Jauvin (2003), A Neural Probabilistic Language Model

Large language models: same recipe, bigger

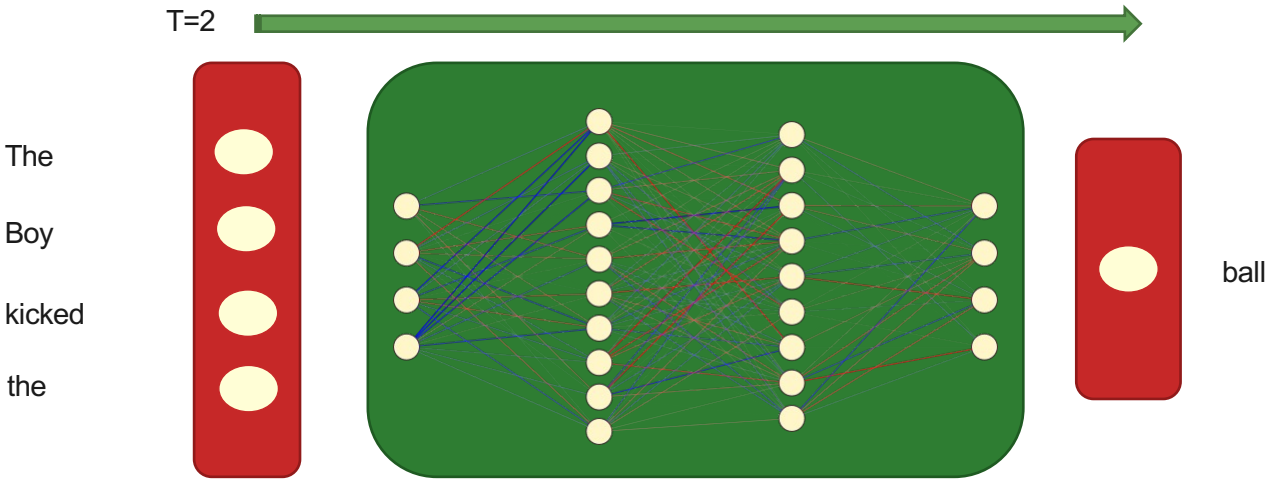
Early neural LM (2003)	Large language model (today)
looks at about 3 words	looks at thousands of words
1 hidden layer	dozens of Transformer layers
millions of numbers	billions of numbers

The Transformer is the architecture inside. We return to it next week.

Conceptual view of neural language model



Conceptual view of neural language model

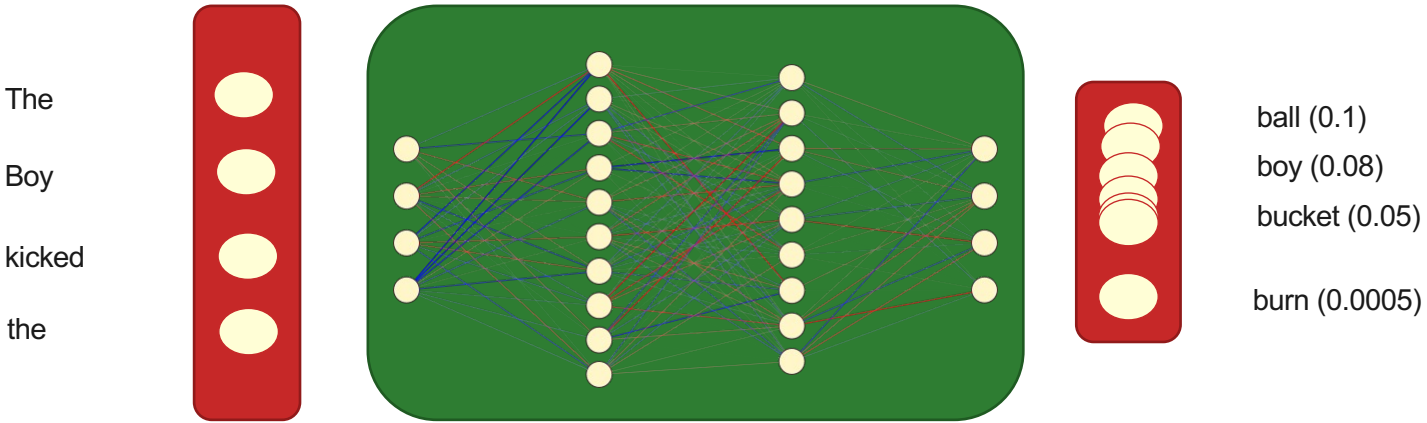


Input
(string of text)

Model
(parameterized math functions)

Output
(text, one token at a time)

Conceptual view of neural language model

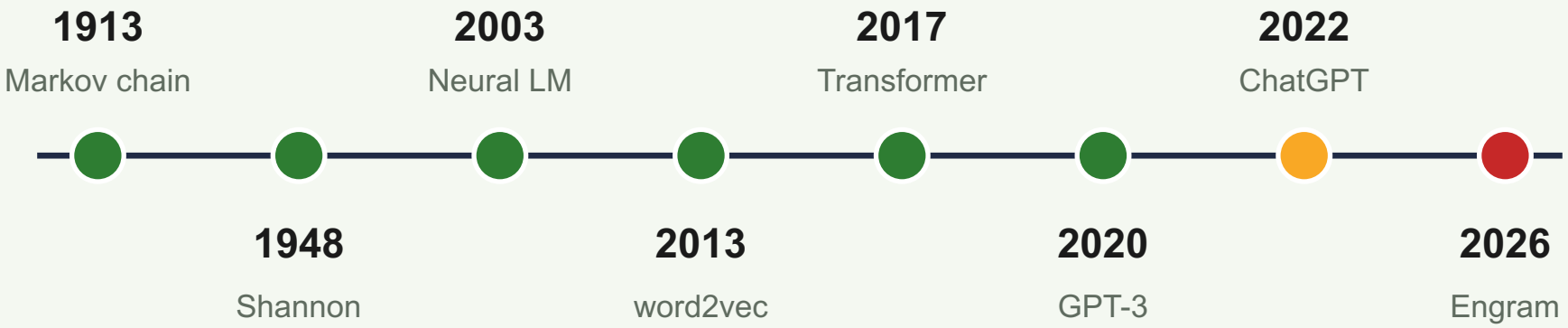


Input
(string of text)

Model
(parameterized math functions)

Output
(text, one token at a time)

From counting to ChatGPT



Over 100 years of predicting the next word

Language models got big, fast

117M	1.5B	175B
GPT-1	**GPT-2**	**GPT-3**
2018 | parameters	2019 | parameters	2020 | parameters

About 1,500 times bigger in two years

Advantages of n-gram Language Models

Simple: just counting and dividing, easy to explain and debug.

Fast and cheap: trains in seconds on a laptop, no GPU needed.

Transparent: every probability can be traced back to real counts.

Still useful: a strong baseline, and back inside LLMs as lookup memory (Engram, 2026).

Disadvantages of n-gram Language Models

Sparsity: most n-grams are never seen; 64% of BBC test trigrams were new.

Short memory: a trigram cannot connect “ring” back to “marry”.

No generalisation: “cat” teaches it nothing about “dog”.

Smoothing is tricky: crude add-one smoothing made our bigram worse than a unigram.

Final Conclusions

- A language model predicts the next word and scores how likely a sentence is.
- Statistical models do this by counting n-grams and dividing.
- Data runs out: unseen n-grams get zero unless we smooth or interpolate.
- Perplexity measures how surprised a model is by new text. Lower is better.
- Neural models learn word vectors, so they generalise to things they have never seen.

Lab: build your own language model

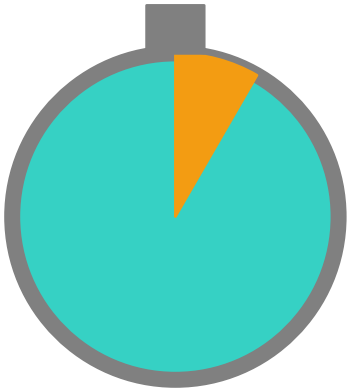
Work through the Week 8 Colab notebook

Upload the BBC corpus and rerun every part

Try the “Your turn” questions at the end

Continue with the assignment





5 minute break

Next week: Applying LLMs

From predicting words to building with them

Interactive Perplexity Demo